MINI PROJECT REPORT

CAMPUS ROUTE FINDER USING GRAPHS

Submitted By: Dipanshu Sharma
Roll Number: 2556451
Course: BTech CSE A
Subject: Data Structures and Algorithms

1. INTRODUCTION

The Campus Route Finder is a mini project developed using Graphs in Data Structures and Algorithms. It represents campus locations as vertices and paths as edges. Breadth-First Search (BFS) finds a shortest route between two locations. Depth-First Search (DFS) finds reachable locations and helps identify disconnected locations through connected components. The graph uses an adjacency list and an adjacency matrix and is implemented in C with a menu-driven interface.

2. OBJECTIVES

• Understand graph concepts and representations.
• Represent campus locations and paths using an adjacency list and adjacency matrix.
• Find a route with the minimum number of edges using BFS.
• Find reachable locations using DFS.
• Identify disconnected locations and connected components.
• Compare time and space complexity for both representations.
• Verify the program using test cases.

3. PROBLEM STATEMENT

Finding a route between campus locations can be difficult when several paths are available or some places are disconnected. This project models the campus as a graph, accepts a source and destination, displays a minimum-edge route when one exists, and reports when no route is available.

4. SOFTWARE AND HARDWARE REQUIREMENTS

Software: C, GCC compiler, VS Code, Windows/Linux/macOS.
Hardware: PC/Laptop, basic memory and storage, display device.

5. DATA STRUCTURE & ALGORITHMS USED

The campus graph is undirected and unweighted. Vertices represent locations and edges represent direct paths.

No.	Location	No.	Location
0	Main Gate	5	Auditorium
1	Library	6	Parking
2	Block 1	7	Lab
3	Block 2	8	Sports Ground (Isolated)
4	Cafeteria		

BFS: Uses a FIFO queue to explore nodes level by level and finds a minimum-edge path in an unweighted graph.
DFS: Traverses deeply along paths using recursion to test reachability and compute connected components.

6. TIME & SPACE COMPLEXITY ANALYSIS

Operation	Adjacency List	Adjacency Matrix
BFS time	O(V + E)	$O(V^2)$
DFS time	O(V + E)	$O(V^2)$
Storage	O(V + E)	$O(V^2)$
Check edge (u,v)	O(degree(u))	O(1)

7. TEST CASES MATRIX

Test	Description	Input	Expected output	Result
1	Direct route	Main Gate to Library	Main Gate → Library	PASS

Test	Description	Input	Expected output	Result
2	Multi-stop route	Main Gate to Auditorium	Main Gate → Library → Cafeteria → Auditorium	PASS
3	Unreachable node	Main Gate to Sports Ground	No route	PASS
4	Self route	Block 1 to Block 1	Block 1	PASS

8. COMPLETE C SOURCE CODE IMPLEMENTATION

C SOURCE CODE — CAMPUS ROUTE FINDER

Formatted C source code. Statements and blocks are separated onto readable lines.

```
#include <stdio.h>
#include <stdlib.h>
#include <stdbool.h>

#define MAX_NODES 9

typedef struct Node {
    int dest;
    struct Node *next;
} Node;

typedef struct Graph {
    int numVertices;
    Node *adjLists[MAX_NODES];
    int adjMatrix[MAX_NODES][MAX_NODES];
} Graph;

char *locations[MAX_NODES] = {
    "Main Gate", "Library", "Block 1", "Block 2",
    "Cafeteria", "Auditorium", "Parking", "Lab", "Sports Ground"
};

Node *createNode(int dest) {
    Node *node = malloc(sizeof(Node));
    if (node == NULL) {
        fprintf(stderr, "Memory allocation failed.\\n");
        exit(EXIT_FAILURE);
    }
    node->dest = dest;
    node->next = NULL;
    return node;
}

Graph *createGraph(int vertices) {
    Graph *graph = malloc(sizeof(Graph));
    if (graph == NULL) {
        fprintf(stderr, "Memory allocation failed.\\n");
        exit(EXIT_FAILURE);
    }
    graph->numVertices = vertices;
    for (int i = 0; i < vertices; i++) {
        graph->adjLists[i] = NULL;
        for (int j = 0; j < vertices; j++)
            graph->adjMatrix[i][j] = 0;
    }
    return graph;
}

void addEdge(Graph *graph, int src, int dest) {
    Node *node = createNode(dest);
    node->next = graph->adjLists[src];
    graph->adjLists[src] = node;

    node = createNode(src);
    node->next = graph->adjLists[dest];
    graph->adjLists[dest] = node;

    graph->adjMatrix[src][dest] = 1;
    graph->adjMatrix[dest][src] = 1;
}
```

C SOURCE CODE — CAMPUS ROUTE FINDER (continued)

```
void bfsShortestPath(Graph *graph, int start, int target) {
    bool visited[MAX_NODES] = {false};
    int parent[MAX_NODES], queue[MAX_NODES];
    int front = 0, rear = 0;

    for (int i = 0; i < MAX_NODES; i++)
        parent[i] = -1;

    visited[start] = true;
    queue[rear++] = start;

    while (front < rear) {
        int current = queue[front++];
        if (current == target)
            break;

        for (Node *temp = graph->adjLists[current];
             temp != NULL; temp = temp->next) {
            int adjacent = temp->dest;
            if (!visited[adjacent]) {
                visited[adjacent] = true;
                parent[adjacent] = current;
                queue[rear++] = adjacent;
            }
        }
    }

    if (!visited[target]) {
        printf("No route exists between %s and %s.\\n",
               locations[start], locations[target]);
        return;
    }

    int path[MAX_NODES], length = 0, current = target;
    while (current != -1) {
        path[length++] = current;
        current = parent[current];
    }

    printf("Shortest Route (BFS): ");
    for (int i = length - 1; i >= 0; i--)
        printf("%s%s", locations[path[i]], i ? " -> " : "\\n");
    printf("Number of edges: %d\\n", length - 1);
}

void dfsTraversal(Graph *graph, int vertex, bool visited[]) {
    visited[vertex] = true;
    printf("%s ", locations[vertex]);

    for (Node *temp = graph->adjLists[vertex];
         temp != NULL; temp = temp->next) {
        if (!visited[temp->dest])
            dfsTraversal(graph, temp->dest, visited);
    }
}

void showConnectedComponents(Graph *graph) {
    bool visited[MAX_NODES] = {false};
    int count = 0;
```

C SOURCE CODE — CAMPUS ROUTE FINDER (continued)

```
    printf("\\n--- CONNECTED COMPONENTS ---\\n");

    for (int v = 0; v < graph->numVertices; v++) {
        if (!visited[v]) {
            printf("Component %d: ", ++count);
            dfsTraversal(graph, v, visited);
            printf("\\n");
        }
    }
    printf("Total connected components: %d\\n", count);
}

int main(void) {
    Graph *graph = createGraph(MAX_NODES);

    addEdge(graph, 0, 1); /* Main Gate - Library */
    addEdge(graph, 0, 6); /* Main Gate - Parking */
    addEdge(graph, 1, 2); /* Library - Block 1 */
    addEdge(graph, 1, 4); /* Library - Cafeteria */
    addEdge(graph, 2, 3); /* Block 1 - Block 2 */
    addEdge(graph, 2, 7); /* Block 1 - Lab */
    addEdge(graph, 3, 5); /* Block 2 - Auditorium */
    addEdge(graph, 4, 5); /* Cafeteria - Auditorium */
    addEdge(graph, 6, 7); /* Parking - Lab */

    int choice, source, destination;
    do {
        printf("\\n=== CAMPUS ROUTE FINDER ===\\n");
        printf("1. Display Locations\\n2. BFS Shortest Route\\n");
        printf("3. DFS Reachability\\n4. Connected Components\\n0. Exit\\n");
        printf("Enter choice: ");

        if (scanf("%d", &choice) != 1)
            break;

        switch (choice) {
            case 1:
                for (int i = 0; i < MAX_NODES; i++)
                    printf("%d. %s\\n", i, locations[i]);
                break;
            case 2:
                printf("Enter source (0-8) and destination (0-8): ");
                if (scanf("%d %d", &source, &destination) != 2 ||
                    source < 0 || source >= MAX_NODES ||
                    destination < 0 || destination >= MAX_NODES) {
                    printf("Invalid location number.\\n");
                    break;
                }
                bfsShortestPath(graph, source, destination);
                break;
            case 3: {
                bool visited[MAX_NODES] = {false};
                printf("Enter starting location (0-8): ");
                if (scanf("%d", &source) != 1 ||
                    source < 0 || source >= MAX_NODES) {
                    printf("Invalid location number.\\n");
                    break;
                }
                printf("Reachable locations from %s: ", locations[source]);
                dfsTraversal(graph, source, visited);
```

C SOURCE CODE — CAMPUS ROUTE FINDER (continued)

```
                printf("\\n");
                break;
            }
            case 4:
                showConnectedComponents(graph);
                break;
            case 0:
                printf("Exiting Campus Route Finder.\\n");
                break;
            default:
                printf("Invalid menu choice.\\n");
        }
    } while (choice != 0);

    return 0;
}
```

9. ASSESSMENT RUBRIC (20 MARKS)

Criteria	Maximum Marks
Data Structure Design & Representation	4
Core Algorithms & Operations (BFS/DFS)	4
Implementation & Working Output	4
Project Report & Screenshots	2
GitHub Submission	1
Viva / Project Presentation	5